MATERIAŁ ROBOCZY

Wstępne ustalenia do pracy inżynierskiej

System generowania grafów infrastruktury energetycznej z wykorzystaniem modeli LLM

Dokument do rozmowy z promotorem
Założenia mogą ulec zmianie po konsultacji

21 października 2026

Charakter dokumentu

To robocze podsumowanie pomysłów z początku pracy. Dokument nie opisuje zatwierdzonej architektury ani ostatecznego zakresu implementacji. Ma pomóc podczas spotkania ustalić, które elementy warto zachować, uprościć albo zmienić.

Główna idea

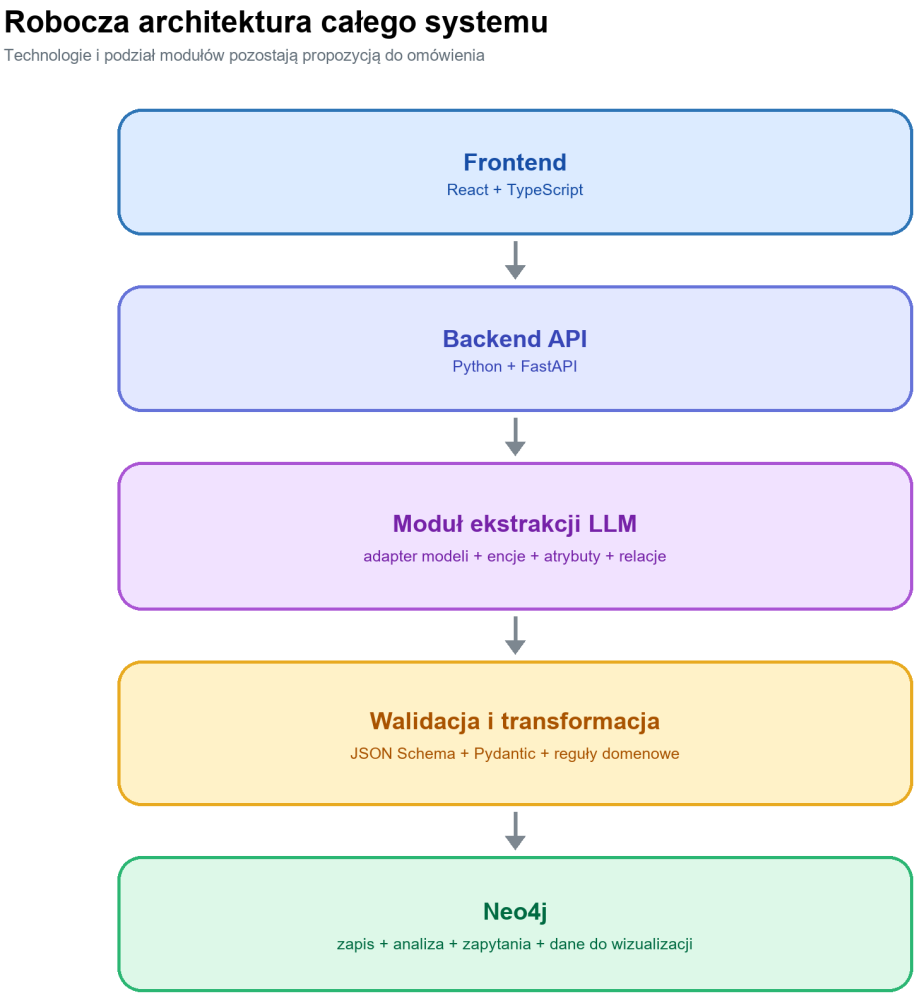
System ma przyjmować opis tekstowy infrastruktury energetycznej, rozpoznawać obiekty oraz relacje, przekształcać wynik do kontrolowanej struktury i zapisywać go w bazie grafowej. Użytkownik powinien następnie móc obejrzeć graf i wykonywać podstawowe zapytania lub analizy.

Co jest ustalone, a co pozostaje propozycją

Element	Aktualny status	Uwagi
LLM przekształca opis w encje i relacje	główna idea pracy	To rdzeń proponowanego systemu.
Grafowa reprezentacja infrastruktury	główna idea pracy	Szczegółowy model wymaga zatwierdzenia.
Neo4j	wstępna propozycja	Pasuje do grafu właściwościowego, ale wybór nie jest jeszcze ostateczny.
Python i FastAPI	wstępna propozycja	Możliwy backend prototypu.
React i TypeScript	wstępna propozycja	Możliwy frontend, jeśli powstanie osobny interfejs webowy.
Modele przez API lub Ollama	wstępna propozycja	Architektura powinna umożliwiać wymianę modelu.
pandapower i case14	przykład do analizy	Nie podjęto decyzji, że będzie to główne źródło danych.

Robocza architektura całego systemu

Poniższy układ pokazuje pełny przepływ od interfejsu użytkownika do bazy grafowej. Jest to propozycja organizacji prototypu, a nie zamknięta decyzja technologiczna.



Rysunek 1 Robocza architektura aplikacji

Frontend odpowiadałby za wprowadzanie opisu i prezentację wyników. Backend koordynowałby wywołanie modelu, walidację danych i komunikację z Neo4j. Moduł LLM nie otrzymywałby bezpośredniego dostępu do bazy.

Rola poszczególnych części

Część	Możliwa technologia	Odpowiedzialność
Frontend	React + TypeScript	Opis sieci, wybór modelu, podgląd grafu, błędy i eksport.
Backend API	Python + FastAPI	Sterowanie procesem, obsługa żądań, sesji i komunikacji między modułami.
Adapter modeli	API dostawcy lub Ollama	Wspólny interfejs umożliwiający podmianę modelu bez zmiany reszty systemu.
Ekstrakcja LLM	prompt + structured output	Rozpoznanie encji, atrybutów i relacji z opisu.
Walidacja	JSON Schema + Pydantic	Sprawdzenie formatu, typów, jednostek, identyfikatorów i relacji.
Transformacja	Python	Odwzorowanie zweryfikowanej struktury na węzły i relacje grafu.
Baza grafowa	Neo4j	Przechowywanie, zapytania, analiza topologii i dane do wizualizacji.

Główny przepływ

- Użytkownik wprowadza opis sieci w interfejsie.
- Backend przekazuje opis, schemat odpowiedzi i instrukcję do wybranego modelu.
- LLM zwraca encje, właściwości i relacje w ustrukturyzowanym formacie.
- Walidator sprawdza wynik i przekazuje użytkownikowi błędy wymagające poprawy.
- Poprawne dane są mapowane na graf i zapisywane w Neo4j.
- Frontend pobiera graf lub wyniki zapytań i prezentuje je użytkownikowi.

Możliwe źródła danych i graf referencyjny

Do oceny jakości LLM potrzebny będzie graf wzorcowy albo ręcznie przygotowany zbiór odpowiedzi. Sposób jego uzyskania nie został jeszcze ustalony. pandapower jest jedną z możliwych opcji, ponieważ udostępnia sieci testowe w ustrukturyzowanych tabelach, lecz można rozważyć również inne zbiory lub własne przykłady.

Możliwość	Zastosowanie	Otwarte kwestie
pandapower	Automatyczne utworzenie grafu referencyjnego z tabel DataFrame	Wybór sieci, wersji biblioteki i zakresu parametrów
Inne sieci testowe	Sprawdzenie, czy podejście działa poza jednym przykładem	Format danych i możliwość automatycznego mapowania
Dane rzeczywiste	Ocena na opisach bliższych praktyce	Dostępność, kompletność i kwestie poufności
Przykłady przygotowane ręcznie	Małe scenariusze kontrolowane na początku projektu	Nakład pracy i ryzyko małej reprezentatywności

Dotychczas sprawdzony przykład case14

W ramach rozpoznania odczytano strukturę pandapower case14. Wynik pokazuje, że biblioteka nadaje się do eksperymentalnego mapowania tabel na graf, ale nie przesądza o jej wykorzystaniu w końcowym systemie.

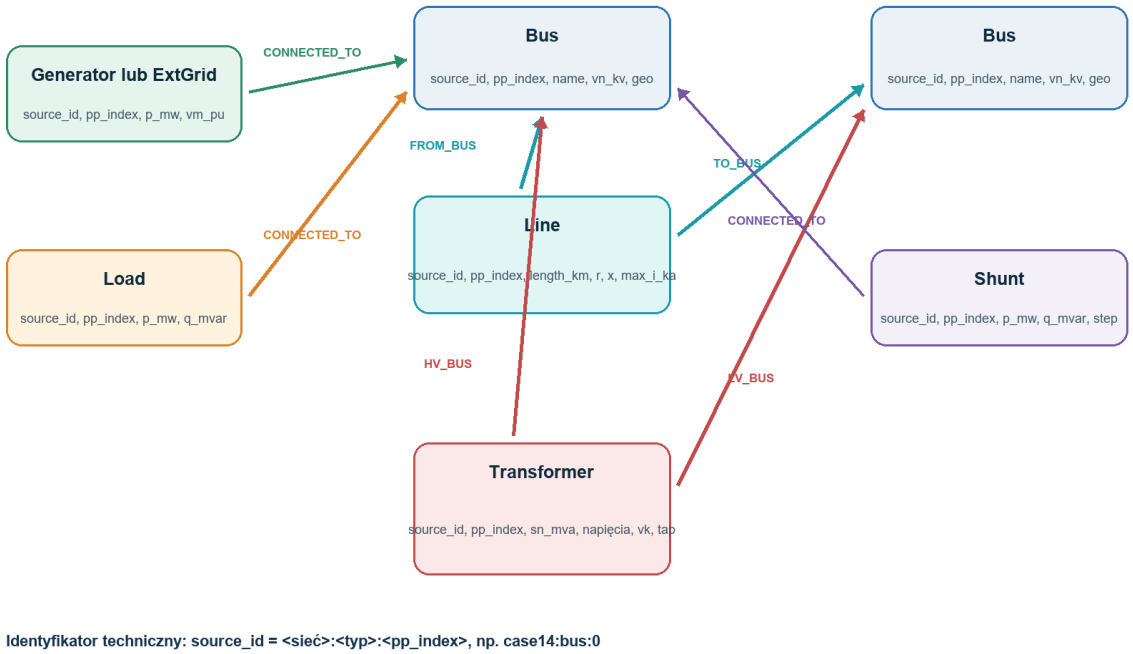
Tabela	Liczba rekordów	Możliwy węzeł grafu
net.bus	14	Bus
net.line	15	Line
net.trafo	5	Transformer
net.gen	4	Generator
net.load	11	Load

Ważna obserwacja techniczna: indeks szyny w DataFrame ma wartości 0-13, natomiast pole name zawiera oznaczenia 1-14. Jeśli pandapower zostanie wybrane, relacje należy budować na podstawie indeksów, a name traktować jako właściwość opisową.

Wstępny model grafu

Poniższy schemat jest punktem startowym do rozmowy. Linie i transformatory zostały przedstawione jako osobne węzły, ponieważ ułatwia to przechowywanie ich parametrów oraz porównywanie elementów wykrytych przez LLM. Możliwy jest też prostszy wariant, w którym część elementów stanowi relacje z właściwościami.

Wstępny model grafu właściwościowego



Rysunek 2 Jedna z możliwych reprezentacji grafowej

Przykładowe typy

Węzły	Relacje
Bus, Line, Transformer	FROM_BUS, TO_BUS
Generator, ExternalGrid	CONNECTED_TO
Load, Shunt	HV_BUS, LV_BUS

Przykładowe zapytania do grafu

Po zapisaniu grafu użytkownik mógłby zadawać pytania o jego elementy i połączenia. Poniższe propozycje pomagają określić minimalny zakres funkcji analitycznych prototypu. Ich dokładna postać zależy od uzgodnionego schematu grafu.

Pytanie użytkownika	Oczekiwany wynik	Zastosowanie
Ile jest elementów każdego typu?	Liczba szyn, linii, transformatorów, źródeł i odbiorów.	Szybki przegląd zawartości grafu.
Co jest podłączone do wybranej szyny?	Sąsiednie linie, transformatory, generatory i odbiory.	Analiza otoczenia wskazanego elementu.
Które szyny łączy dana linia lub transformator?	Dwa końce wybranego elementu oraz jego właściwości.	Sprawdzenie relacji wyekstrahowanych z tekstu.
Jakie elementy leżą na trasie między dwiema szynami?	Ścieżka połączeń albo informacja o jej braku.	Analiza topologii sieci.
Które elementy mają niepełne dane?	Lista obiektów bez wymaganych atrybutów lub połączeń.	Kontrola jakości danych przed analizą.
Z którego fragmentu opisu pochodzi element?	Obiekt i przypisany mu fragment tekstu, jeśli będzie przechowywany.	Wyjaśnienie wyniku i ręczna korekta.

Dwa orientacyjne przykłady Cypher

Zliczenie elementów według etykiety węzła:

```
MATCH (n)
RETURN labels(n)[0] AS typ, count(*) AS liczba
ORDER BY typ
```

Linie podłączone do wskazanej szyny (parametr `$bus_id`):

```
MATCH (b:Bus {id: $bus_id})<-[:FROM_BUS|TO_BUS]-(l:Line)
RETURN l.id AS linia, l.length_km AS dlugosc_km
```

Przykłady zakładają, że identyfikator jest przechowywany jako właściwość id, a linia jest węzłem połączonym relacjami FROM_BUS i TO_BUS z szynami. Po decyzji o modelu grafu trzeba doprecyzować składnię oraz zakres zapytań dostępnych w interfejsie.

Wstępny format wyniku LLM

Model może zwracać wynik JSON zawierający listę obiektów i relacji. Dokładna struktura będzie zależeć od zatwierdzonego modelu grafu. Backend powinien przyjmować wyłącznie dane zgodne ze schematem oraz zgłaszać brakujące lub sprzeczne informacje.

```
{
  "nodes": [
    {"id": "B1", "type": "Bus", "properties": {"vn_kv": 110}},
    {"id": "L1", "type": "Line", "properties": {"length_km": 10}}
  ],
  "relationships": [
    {"type": "FROM_BUS", "from": "L1", "to": "B1"}
  ]
}
```

Przykładowe reguły walidacji

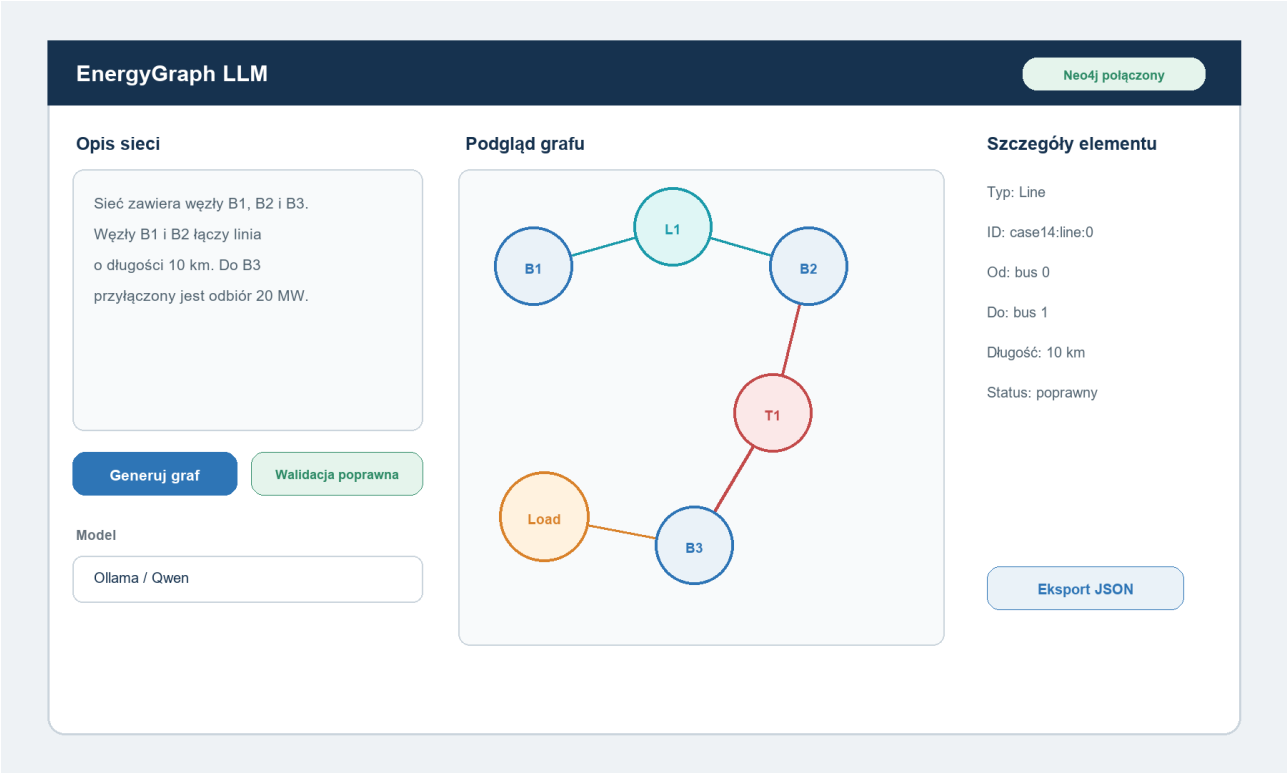
- Każdy obiekt ma identyfikator i typ należący do uzgodnionego słownika.
- Relacja wskazuje obiekty, które istnieją w tej samej odpowiedzi.
- Dany typ relacji łączy tylko dozwolone typy węzłów.
- Właściwości liczbowe mają określone jednostki i dopuszczalne zakresy.
- Brak informacji w opisie jest oznaczany jako brak, a nie uzupełniany domysłem modelu.

Co pozostaje do ustalenia

- Czy jeden schemat ma obsługiwać różne rodzaje sieci energetycznych.
- Jak przechowywać fragment tekstu, z którego pochodzi obiekt lub relacja.
- Czy zapisywać poziom pewności modelu i informacje o użytym modelu lub promptach.
- Jak obsługiwać korekty użytkownika przed zapisem grafu.

Możliwy widok aplikacji

Interfejs nie musi być rozbudowany. W pierwszym prototypie wystarczy ekran pozwalający wprowadzić opis, uruchomić ekstrakcję, zobaczyć wynik walidacji oraz obejrzeć wygenerowany graf. Makieta pokazuje jedynie możliwe rozmieszczenie funkcji.



Rysunek 3 Robocza makieta interfejsu

Obszar	Możliwy zakres
Opis	Pole tekstowe, wybór modelu i uruchomienie procesu
Graf	Podgląd węzłów i relacji oraz filtrowanie typów
Szczegóły	Właściwości zaznaczonego elementu i źródło informacji
Walidacja	Lista ostrzeżeń, błędów i elementów wymagających zatwierdzenia

Wstępna koncepcja ewaluacji

Ewaluacja powinna porównywać wynik LLM z oczekiwaną strukturą, niezależnie od tego, czy graf referencyjny powstanie z pandapower, innych danych czy zostanie przygotowany ręcznie.

Obszar	Przykładowa miara	Pytanie
Węzły	precision, recall, F1	Czy model wykrył właściwe obiekty?
Relacje	precision, recall, F1	Czy połączenia i ich kierunki są poprawne?
Atrybuty	exact match lub błąd liczbowy	Czy wartości i jednostki zgadzają się z opisem?
Format	odsetek poprawnych odpowiedzi	Jak często wynik przechodzi walidację bez naprawy?
Efektywność	czas, tokeny, koszt	Jak drogie i szybkie jest pojedyncze przetworzenie?

Możliwe porównania

- Modele lokalne i modele dostępne przez API.
- Prompt bez przykładów i prompt zawierający kilka przykładów.
- Zwykła odpowiedź tekstowa i wymuszony structured output.
- Opisy proste, złożone, niepełne i niejednoznaczne.

To również wymaga uzgodnienia

Trzeba określić, ile modeli i wariantów promptów porównać oraz jakie dane zapewnią wystarczającą wiarygodność wyników w pracy inżynierskiej.

Proponowane następne kroki

Krok	Cel
1 Uzgodnienie zakresu	Określenie, co dokładnie ma znaleźć się w działającym prototypie.
2 Wybór modelu grafu	Zatwierdzenie typów węzłów, relacji i minimalnych właściwości.
3 Wybór danych	Decyzja, czy wykorzystać pandapower, inne sieci, dane ręczne lub ich połączenie.
4 Mały eksperyment	Opis kilku elementów, ekstrakcja do JSON i ręczne porównanie wyniku.
5 Graf w Neo4j	Zapis zweryfikowanego przykładu i wykonanie podstawowych zapytań.
6 Prototyp aplikacji	Połączenie interfejsu, backendu, modelu i bazy.
7 Ewaluacja	Przygotowanie większego zbioru i porównanie wybranych wariantów.

Pytania na spotkanie

- Czy taka ogólna architektura systemu odpowiada zakresowi pracy?
- Czy frontend webowy jest potrzebny, czy wystarczy prostszy interfejs demonstracyjny?
- Czy Neo4j ma pozostać docelową bazą grafową?
- Czy linie i transformatory reprezentować jako węzły, czy jako relacje z właściwościami?
- Jakie źródło danych będzie odpowiednie do utworzenia grafu referencyjnego?
- Czy warto dalej analizować pandapower i case14, czy wybrać inne dane?
- Jak szeroka powinna być ewaluacja modeli i promptów?
- Czy obliczenia elektroenergetyczne wchodzą w zakres, czy system ma skupić się na ekstrakcji i grafach?

Podsumowanie robocze

Najbardziej stabilnym elementem pomysłu jest przepływ od opisu tekstowego do grafu: LLM rozpoznaje informacje, backend je sprawdza i zapisuje w bazie grafowej. Pozostałe decyzje, w tym konkretne technologie, źródła danych, szczegółowy schemat grafu oraz zakres interfejsu, powinny zostać omówione z promotorem.

pandapower i case14 są na tym etapie przykładem, który pomógł sprawdzić, jak mogą wyglądać ustrukturyzowane dane infrastruktury. Nie oznacza to jeszcze wyboru tej biblioteki jako podstawy całej pracy.

Materiały pomocnicze

- pandapower Power System Test Cases: https://pandapower.readthedocs.io/en/v3.5.6/networks/power_system_test_cases.html
- pandapower repository: https://github.com/e2nIEE/pandapower
- Neo4j graph database concepts: https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/